MSE 암호화 분석 문서

개요

MSE(Message Stream Encryption)는 BitTorrent 프로토콜의 암호화 확장으로, 수동적 도청을 방지하고 프로토콜 식별을 어렵게 만드는 것을 목표로 합니다. Rain BitTorrent 클라이언트는 완전한 MSE 구현을 제공하여 네트워크 트래픽 분석과 차단을 우회할 수 있도록 합니다.

주요 특징

- Diffie-Hellman 키 교환을 통한 안전한 키 생성
- RC4 스트림 암호화와 PlainText 모드 지원
- 양방향 호환성 (암호화/비암호화 피어 모두 지원)
- 초기 페이로드와 함께 핸드셰이크 수행
- 빠른 암호화 방법으로 성능 최적화

암호화 협상 과정

1. MSE 스트림 구조

MSE는 기존 연결을 래핑하여 투명한 암호화를 제공합니다 (`internal/mse/mse.go`):

```
// Stream은 자동으로 암호화/복호화를 수행하는 io.ReadWriter 래퍼입니다.
type Stream struct {
raw io.ReadWriter // 원본 연결
r *cipher.StreamReader // 복호화 리더
```

```go
w *cipher.StreamWriter // 암호화 라이터
r2 io.Reader // 최종 리더 (초기 페이로드 포함)
}

// 새 스트림 생성
func NewStream(rw io.ReadWriter) *Stream {
return &Stream{raw: rw}
}
```

2. 암호화 방법 정의

MSE는 두 가지 암호화 방법을 지원합니다:

```go
// CryptoMethod는 32비트 비트필드로 각 비트가 하나의 암호화 방법을 나타냅니다.
type CryptoMethod uint32

const (
PlainText CryptoMethod = 1 << iota // 평문 (암호화 없음)
RC4 // RC4 스트림 암호화
)

func (c CryptoMethod) String() string {
switch c {
case PlainText:
```

```
return "PlainText"
case RC4:
return "RC4"
default:
return "unknown"
}
}
```

3. Diffie-Hellman 키 교환

MSE는 Diffie-Hellman 알고리즘을 사용하여 공유 비밀을 생성합니다:

```
var (
// 1024비트 소수 (RFC 2409에서 정의)
pBytes = []byte{255, 255, 255, 255, /* ... 생략 ... */}
p = new(big.Int) // 소수 P
g = big.NewInt(2) // 생성원 G = 2
)

// 개인키 생성 (160비트 랜덤)
func privateKey() (*big.Int, error) {
b := make([]byte, 20)
_, err := rand.Read(b)
if err != nil {
```

```go
return nil, err
}
var n big.Int
return n.SetBytes(b), nil
}

// 공개키 생성: Y = G^X mod P
func publicKey(private *big.Int) *big.Int {
var n big.Int
return n.Exp(g, private, p)
}

// 키 쌍 생성
func keyPair() (private, public *big.Int, err error) {
private, err = privateKey()
if err != nil {
return
}
public = publicKey(private)
return
}
```

아웃고잉 핸드셰이크 과정

1. 핸드셰이크 시작

클라이언트가 서버에 연결할 때 MSE 핸드셰이크를 시작합니다:

```
func (s *Stream) HandshakeOutgoing(sKey []byte, cryptoProvide CryptoMethod, init
if  cryptoProvide == 0 {
err = errors.New("no crypto methods are provided")
return
}
if  len(initialPayload) > math.MaxUint16 {
err = errors.New("initial payload is too big")
return
}
// ... 핸드셰이크 로직 ...
}
```

2. 4단계 핸드셰이크 프로세스

MSE 핸드셰이크는 4단계로 구성됩니다:

Step 1: A→B: Diffie Hellman Ya, PadA

```
// 키 쌍 생성
Xa, Ya, err := keyPair()
```

```go
if  err != nil {
return
}


// Step 1: 공개키 Ya와 랜덤 패딩 전송
writeBuf.Write(bytesWithPad(Ya)) // 96바이트로 패딩된 공개키
padA, err := padRandom() // 0-511바이트 랜덤 패딩
if  err != nil {
return
}
writeBuf.Write(padA)
_, err = writeBuf.WriteTo(s.raw)
```

Step 2: B→A: Diffie Hellman Yb, PadB

```go
// 상대방 공개키 Yb 수신
b := make([]byte, 96+512)
firstRead, err := io.ReadAtLeast(s.raw, b, 96)
if  err != nil {
return
}
Yb := new(big.Int)
Yb.SetBytes(b[:96])
```

```
// 공유 비밀 계산: S = Yb^Xa mod P
S := Yb.Exp(Yb, Xa, p)


// RC4 암호화 초기화
err = s.initRC4("keyA", "keyB", S, sKey)
```

Step 3: A→B: 인증 및 암호화 협상

```
// 해시 계산
hashS, hashSKey := hashes(S, sKey)


// 암호화된 데이터 구성
writeBuf.Write(hashS) // HASH('req1', S)
writeBuf.Write(hashSKey) // HASH('req2', SKEY) xor HASH('req3', S)
writeBuf.Write(vc) // 검증 상수 (8바이트 0)
_ = binary.Write(writeBuf, binary.BigEndian, cryptoProvide) // 지원하는 암호화 방법
_ = binary.Write(writeBuf, binary.BigEndian, uint16(len(padC))) // 패딩 길이
writeBuf.Write(padC) // 패딩
_ = binary.Write(writeBuf, binary.BigEndian, uint16(len(initialPayload))) // 초기
writeBuf.Write(initialPayload) // 초기 페이로드


// 암호화 (VC부터)
encBytes := writeBuf.Bytes()[40:]
```

```
s.w.S.XORKeyStream(encBytes, encBytes)
```

Step 4: B→A: 암호화 방법 선택

```
// 암호화된 VC 동기화
vcEnc := make([]byte, 8)
s.r.S.XORKeyStream(vcEnc, vc)
err = s.readSync(vcEnc, 616-firstRead)

// 선택된 암호화 방법 수신
err = binary.Read(s.r, binary.BigEndian, &selected)
if  selected == 0 {
err = errors.New("none of the provided methods are accepted")
return
}

// 최종 암호화 설정
s.updateCipher(selected)
s.r2 = s.r
```

인커밍 핸드셰이크 과정

1. 핸드셰이크 수락

서버가 클라이언트 연결을 수락할 때 MSE 핸드셰이크를 처리합니다:

```
func (s *Stream) HandshakeIncoming(
getSKey  func(sKeyHash [20]byte) (sKey []byte),
cryptoSelect  func(provided CryptoMethod) (selected CryptoMethod)) (err  error)
// ... 핸드셰이크 로직 ...
}
```

2. 키 검증과 암호화 선택

```
// Step 3에서 받은 해시로 sKey 검증
req1 := hashInt("req1", S)
err = s.readSync(req1, 628-firstRead)

var  hashRead [20]byte
_, err = io.ReadFull(s.raw, hashRead[:])

// XOR 해제하여 원본 sKey 해시 복원
req3 := hashInt("req3", S)
for  i := 0;  i < sha1.Size;  i++ {
hashRead[i] ^= req3[i]
}
```

```
// sKey 검증
sKey := getSKey(hashRead)
if  sKey == nil {
err = errors.New("invalid SKEY hash")
return
}


// 암호화 방법 선택
selected := cryptoSelect(cryptoProvide)
if  selected == 0 {
err = errors.New("none of the provided methods are accepted")
return
}
```

암호화 방법별 구현

1. RC4 암호화

RC4는 주요 암호화 방법으로 사용됩니다:

```
func (s *Stream) initRC4(encKey, decKey  string, S *big.Int, sKey []byte) error
// 암호화/복호화 키 생성
cipherEnc, err := rc4.NewCipher(rc4Key(encKey, S, sKey))
if  err != nil {
```

```go
return err
}
cipherDec, err := rc4.NewCipher(rc4Key(decKey, S, sKey))
if err != nil {
return err
}
// 초기 1024바이트 폐기 (RC4 보안 강화)
var buf [1024]byte
discard := buf[:]
cipherEnc.XORKeyStream(discard, discard)
cipherDec.XORKeyStream(discard, discard)
// 스트림 래퍼 설정
s.w = &cipher.StreamWriter{S: cipherEnc, W: s.raw}
s.r = &cipher.StreamReader{S: cipherDec, R: s.raw}
return nil
}

// RC4 키 생성: HASH(prefix + S + sKey)
func rc4Key(prefix string, S *big.Int, sKey []byte) []byte {
h := sha1.New()
_, _ = h.Write([]byte(prefix))
_, _ = h.Write(bytesWithPad(S))
_, _ = h.Write(sKey)
```

```go
return h.Sum(nil)
}
```

2. PlainText 모드

암호화를 사용하지 않는 모드입니다:

```go
func (s *Stream) updateCipher(selected CryptoMethod) {
switch selected {
case RC4:
// RC4는 이미 initRC4에서 설정됨
case PlainText:
// 평문 모드: 암호화 없이 그대로 복사
s.r = &cipher.StreamReader{S: plainTextCipher{}, R: s.raw}
s.w = &cipher.StreamWriter{S: plainTextCipher{}, W: s.raw}
}
}

// 평문 암호화 (실제로는 복사만 수행)
type plainTextCipher struct{}

func (plainTextCipher) XORKeyStream(dst, src []byte) {
copy(dst, src)
}
```

핸드셰이크 과정에서의 MSE 통합

1. 아웃고잉 연결에서 MSE

아웃고잉 연결 시 MSE 핸드셰이크를 시도합니다 (`internal/btconn/dial.go`):

```
func Dial(addr net.Addr, dialTimeout, handshakeTimeout time.Duration,
enableEncryption, forceEncryption bool,
ourExtensions [8]byte, ih [20]byte, ourID [20]byte,
stopC chan struct{}) (conn net.Conn, cipher mse.CryptoMethod, ...) {
// BitTorrent 핸드셰이크를 버퍼에 준비
out := bytes.NewBuffer(make([]byte, 0, 68))
err = writeHandshake(out, ih, ourID, ourExtensions)
if enableEncryption {
sKey := make([]byte, 20)
copy(sKey, ih[:]) // InfoHash를 sKey로 사용
// 지원할 암호화 방법 설정
provide := mse.RC4
if !forceEncryption {
provide |= mse.PlainText
}
// MSE 핸드셰이크 시도
encConn := mse.WrapConn(conn)
cipher, err = encConn.HandshakeOutgoing(sKey, provide, out.Bytes())
```

```go
if err != nil {
if forceEncryption {
err = errNotEncrypted
return
}
// 암호화 실패 시 평문으로 재시도
conn.Close()
conn, err = dialer.DialContext(ctx, addr.Network(), addr.String())
// ... 평문 핸드셰이크 수행 ...
}
}
}
```

2. 인커밍 연결에서 MSE

인커밍 연결 시 MSE와 평문을 모두 지원합니다 (`internal/btconn/accept.go`):

```go
func Accept(conn net.Conn, handshakeTimeout time.Duration,
getSKey func(sKeyHash [20]byte) (sKey []byte),
forceEncryption bool, hasInfoHash func([20]byte) bool,
ourExtensions [8]byte, ourID [20]byte) (...) {
isEncrypted := false
// 먼저 평문 핸드셰이크 시도
var buf bytes.Buffer
```

```
reader := io.TeeReader(conn, &buf)
peerExtensions, infoHash, err := readHandshake1(reader)
if  err == errInvalidProtocol && getSKey != nil {
// 평문 실패 시 MSE 핸드셰이크 시도
conn = &rwConn{readWriter{io.MultiReader(&buf, conn), conn}, conn}
mseConn := mse.WrapConn(conn)
err = mseConn.HandshakeIncoming(
getSKey,
func(provided mse.CryptoMethod) (selected mse.CryptoMethod) {
if  provided&mse.RC4 != 0 {
selected = mse.RC4
isEncrypted = true
} else  if (provided&mse.PlainText != 0) && !forceEncryption {
selected = mse.PlainText
}
cipher = selected
return
})
if  err != nil {
return
}
conn = mseConn
peerExtensions, infoHash, err = readHandshake1(conn)
```

```
}
if  forceEncryption && !isEncrypted {
err = errNotEncrypted
return
}
}
```

3. MSE 연결 래퍼

MSE 연결은 표준 net.Conn 인터페이스를 구현합니다 (`internal/mse/conn.go`):

```
// Conn은 Read/Write 메서드에서 암호화/복호화를 수행하는 net.Conn 래퍼입니다.
type Conn struct {
net.Conn
*Stream
}

// 연결 래핑
func  WrapConn(conn net.Conn) *Conn {
return &Conn{
Conn: conn,
Stream: NewStream(conn),
}
}
```

```go
// 암호화된 읽기
func (c *Conn) Read(p []byte) (n int, err error) {
n, err = c.Stream.Read(p)
return
}

// 암호화된 쓰기
func (c *Conn) Write(p []byte) (n int, err error) {
n, err = c.Stream.Write(p)
return
}
```

성능 고려사항

1. 빠른 암호화 방법

MSE는 성능을 위해 빠른 암호화 방법을 선택합니다:

- **RC4**: 빠른 스트림 암호화, 하드웨어 가속 지원
- **SHA-1**: 해시 함수로 널리 지원됨
- **1024비트 DH**: 적절한 보안과 성능의 균형

2. 메모리 효율성

```go
// 96바이트로 패딩하여 고정 크기 사용
```

```go
func bytesWithPad(key *big.Int) []byte {
b := key.Bytes()
pad := 96 - len(b)
if pad > 0 {
b = make([]byte, 96)
copy(b[pad:], key.Bytes())
}
return b
}

// 랜덤 패딩으로 트래픽 분석 방해
func padRandom() ([]byte, error) {
padLen, err := rand.Int(rand.Reader, big.NewInt(512))
if err != nil {
return nil, err
}
b := make([]byte, int(padLen.Int64()))
_, err = rand.Read(b)
return b, err
}
```

3. 동기화 최적화

```go
// 효율적인 동기화 포인트 검색
func (s *Stream) readSync(key []byte, max int) error {
var readBuf bytes.Buffer
if _, err := io.CopyN(&readBuf, s.raw, int64(len(key))); err != nil {
return err
}
max -= len(key)
for {
if bytes.Equal(readBuf.Bytes(), key) {
return nil // 동기화 포인트 발견
}
if max <= 0 {
return errors.New("sync point is not found")
}
// 슬라이딩 윈도우로 효율적 검색
if _, err := io.CopyN(&readBuf, s.raw, 1); err != nil {
return err
}
max--
if _, err := io.CopyN(io.Discard, &readBuf, 1); err != nil {
return err
}
```

```
}

}
```

보안 고려사항

1. 제한된 보안 목표

MSE는 다음과 같은 제한된 보안을 제공합니다:

- **수동적 도청 방지**: 네트워크 트래픽 분석 어려움
- **프로토콜 식별 방해**: BitTorrent 트래픽 탐지 어려움
- **포트 스캐닝 방해**: 약한 공유 비밀로 연결 완료 필요

2. 보안 제한사항

- **피어 인증 없음**: 상대방 신원 확인 불가
- **데이터 무결성 미보장**: 변조 탐지 불가
- **MITM 공격 취약**: 중간자 공격에 제한적 보호만 제공

3. InfoHash 기반 인증

```
// InfoHash를 sKey로 사용하여 토렌트별 인증
func (t *torrent) getSKey(sKeyHash [20]byte) []byte {
if  bytes.Equal(sKeyHash[:], t.sKeyHash[:]) {
return  t.infoHash[:] // InfoHash 반환
}
return  nil  // 잘못된 해시면 nil 반환
```

```
}
```

관련 문서

선행 지식

- 04-01-networking-packages-analysis.md - 피어 연결과 핸드셰이크 기초
- 01-system-architecture.md - MSE가 전체 시스템에서 차지하는 위치

관련 주제

- 03-01-torrent-lifecycle-process.md - 핸드셰이크 과정에서의 MSE 역할
- 06-01-performance-optimization-analysis.md - 암호화 성능 최적화

후속 학습

- 07-01-library-usage-guide.md - MSE 설정 및 사용법
- 06-03-error-handling-recovery-analysis.md - 암호화 오류 처리

> **네비게이션:** 이전: PEX 프로토콜 분석 | 목차 | 다음: WebSeed 지원 분석